

HoloTalking — Code Explained

A complete walkthrough of the AI talking-avatar → holographic-fan pipeline.

1. What this project does

HoloTalking turns **text or a voice message** into a **short lip-synced video of an avatar speaking Vietnamese**, then pushes that video onto a **3D holographic LED fan** (the "holofan").

The full chain, one interaction:

You type / speak in a web page

▼

(1) the browser sends it to the backend

FastAPI backend (server_api.py, runs on a GPU machine "i86")

■

■ (2) AI reply Gemini or Claude turns your message into a spoken answer

■ (3) Text-to-speech VieNeu or VoxCPM makes a Vietnamese voice .wav

■ (4) Talking head SadTalker and/or Wav2Lip animate the avatar to the audio

■ (5) Result saved result/latest_result.mp4

■

▼

(6) backend pushes the .mp4 over a TCP socket (port 9999)

holofan_autopush.py (runs on the machine next to the fan)

■

▼

(7) it drives the "Holoscope" desktop app to upload the clip

Holoscope app ■■(fan's Wi-Fi)■■■ Holographic fan spins & shows the avatar

There are **three programs** and they usually run on **three machines**:

Program	File(s)	Runs on	Job
Backend	backend/SadTalker/server_api.py (+ helpers)	GPU box (i86, remote)	generate the video
Frontend	frontend/ (React)	any browser	the UI you click
Fan bridge	holofan_autopush.py	machine near the fan (Mac/Windows)	get the video onto the fan

Because the GPU box is reached over the internet (SSH) and the fan only speaks its own Wi-Fi, a fair amount of the project is **plumbing** to connect those two worlds.

2. Repository layout

app/
 backend/
 SadTalker/
 server_api.py ← THE BACKEND. FastAPI app, ~2000 lines. Read this first.
 sadtalker_engine.py ← keeps SadTalker's models loaded in-process (speed)
 inference.py ← SadTalker's original command-line entry point
 src/utils/
 wav2lip_processor.py ← thin wrapper that shells out to wav2lip
 wav2lip_inference.py ← the wav2lip model runner (from upstream)
 src/... ← upstream SadTalker model code (face3d, facerender, ...)
 models/ ← upstream wav2lip model definitions
 face_detection/... ← upstream face detector
 socket_video_server.py ← the TCP "push" server on port 9999
 socket_video_client.py ← a minimal receiver (reference)
 holofan_test_receiver.py ← a receiver used for latency testing
 frontend/
 src/
 App.jsx ← main React component, all the generation logic
 api.js ← where the backend UI comes from
 components/
 Sidebar/ ← input tabs, voice settings, avatar picker
 VideoArea/ ← the video player + status
 holofan_autopush.py ← receives videos, drives the Holoscope app
 start_holo_mac.sh ← one command: SSH tunnel + Frontend + autopush
 result/ ← latest_result.mp4 lives here

HoloTalking — Code Explained · page 1 of 10

Important distinction: `src/`, `models/`, `face_detection/` are the **original third-party SadTalker and Wav2Lip codebases**. This project barely touches them — it treats them as libraries. The custom work is in `server_api.py`, `sadtalker_engine.py`, `wav2lip_processor.py`, the `socket_video_*` files, the whole `frontend/`, and `holofan_autopush.py`.

3. The backend — `server_api.py`

A single FastAPI application. It is organised as: helpers → model loaders → warm-up → HTTP endpoints.

3.1 Audio & image helpers (lines ~48–265)

Small utilities used by the endpoints.

- **`transcode_to_wav_bytes(raw_bytes)`** — the browser's microphone recorder produces WebM/Opus even when the file is named `.wav`. This runs `ffmpeg` to convert *any* audio blob into real 16 kHz mono WAV. Gemini rejects audio whose declared type doesn't match its contents, so everything is normalised here first.
- **`apply_speed_to_audio(path, speed)`** — speeds up / slows down a WAV in place using `ffmpeg`'s `atempo` filter (keeps the pitch natural). No-op at speed 1.0.
- **`remove_image_background(input, output)`** — makes the avatar background **pure black**, which the holographic fan needs. Tries the local `rembg` library first; falls back to the `remove.bg` web API; if neither is available, just copies the image through.
- **`crop_to_1x1_square(...)`** — face-centred square crop (detects the face with OpenCV's Haar cascade, else centre-crops).
- **`get_cached_avatar_path(image_path)`** — copies the processed avatar to `avatar_cache/<md5>.png`. SadTalker caches its expensive 3D-face analysis next to the source image, keyed by path. By keying the copy on the file's **content hash**, repeat generations against the same avatar reuse that cache instead of recomputing.
- **`clear_results_and_temp_files()`** — wipes `temp_files/` and `result/` at the start of each new avatar so they don't grow forever.

3.2 Model loaders (lines ~272–430)

Each heavy model is loaded **once** and reused. The pattern is always: a module-level `_x_instance = None`, a `get_x()` that loads on first call under a lock, and everything else calls `get_x()`.

- **`_pick_tts_device()`** — returns `"cuda"` if an NVIDIA GPU is present, else `"cpu"`. `mps` (Apple) is deliberately excluded because VoxCPM runs many small sequential ops that are far slower on Apple's GPU than on CPU. Override with `TTS_DEVICE`.
- **`get_voxcpm()`** — loads VoxCPM (a voice-cloning TTS). `optimize=True` (`torch.compile` + a warm-up generation) is only used on CUDA; on CPU it's skipped so the server starts faster.
- **`get_vieneu()`** — loads VieNeu (a native Vietnamese preset-voice TTS). **This was a real bug once:** the old code created `Vieneu()` on every sentence, which re-downloaded the model each time. Now it's cached.
- **`get_elevenlabs_client(api_key)`** — an ElevenLabs client, only if a key is configured.

3.3 Text-to-speech — `synthesize_tts(...)` (lines ~402–494)

The single entry point for "text → `audio_path`". Dispatches by `tts_engine`:

- **"vietneu"** (default) — picks a preset Vietnamese voice by name ("`Thái S`", "`Ng` Lan", ...) and synthesises directly. Fast, fully local.
- **"voxcpm"** — clones a voice. The reference audio comes from, in priority order:
- `custom_voice_ref` — a WAV the user uploaded via `/api/custom_voice/upload`
- `elevenlabs_voice_id` — VieNeu speaks a reference line, then ElevenLabs' speech-to-speech re-voices it into the chosen ElevenLabs timbre (cached in `custom_voices/`)

- `voice_name` — a VieNeu preset's own timbre used directly as the clone reference VoxCPM also accepts a natural-language delivery hint (`voice_style`), prepended in parentheses.

Finally `apply_speed_to_audio()` adjusts tempo.

3.4 Speech-to-text (Claude path only) — lines ~1377–1477

Claude has **no audio input**, so when `AI_PROVIDER=c laude` (or `GEMINI_STT_FIRST=1`), voice messages are transcribed locally first.

- `_get_whisper()` — loads either **faster-whisper** (CTranslate2, ~4× quicker, built-in voice-activity-detection) or the Hugging Face transformers Whisper pipeline as a fallback. Controlled by `STT_ENGINE`, `WHISPER_MODEL`, `STT_DEVICE`.
- `_transcribe_audio(bytes)` — writes the audio to a temp file, then:
- **silence guard**: if the recording's RMS energy is below a threshold, return "" (Whisper hallucinates a fixed "please subscribe to my channel" phrase on silence).
- runs the model with Vietnamese language + greedy decoding + VAD.
- **hallucination filter**: drops the result if it contains known junk phrases.

3.5 The AI brain — reply generation (lines ~1479–1745)

- `_get_anthropic(api_key)` — a cached Anthropic client. Adds the `anthropic-workspace-id` header when `ANTHROPIC_WORKSPACE_ID` is set (workspace-scoped keys require it).
- **Conversation memory** — `load_history`, `append_history`, `clear_history`. Each browser has a `session_id`; the full user/assistant transcript is stored as JSON in `conversations/<session_id>.json`, so the avatar **remembers the whole conversation** across turns and server restarts. Only the last `CLAUDE_HISTORY_TURNS` (default 20) messages are sent to the model.
- `_gemini_reply(system, history, user_text, audio_bytes, ...)` — the Gemini path. Builds `contents` (history + current turn), tries `gemini-2.5-flash-lite` → `gemini-flash-latest` → `gemini-2.5-flash` (fastest first; override with `GEMINI_MODEL`), with `thinkingConfig` disabled and `retryWithout` on a 400. If `audio_bytes` is given, the audio is sent to Gemini directly (native audio understanding).
- `_claude_reply(system, history, user_text, ...)` — the Claude path. Standard Messages API call, `effort` from `CLAUDE_EFFORT`, model from `CLAUDE_MODEL` (default `claude-opus-5`).
- `generate_gemini_response(...)` — the dispatcher (name kept for backwards compatibility). Steps:
- Build the system prompt. `AI_BRIEF=1` forces a single short sentence (good for latency testing); otherwise 1–2 short sentences. The prompt tells the model the reply will be **spoken aloud** — no markdown, no emoji, no parentheticals.
- Resolve the user's turn to text: use `user_message` if given; for audio, transcribe with Whisper (Claude, or `GEMINI_STT_FIRST`) or pass the audio through (Gemini).
- Load conversation memory.
- Call `_claude_reply` or `_gemini_reply` based on `AI_PROVIDER` (default `gemini`).
- On success, append both turns to the on-disk history.
- On any failure, return "" — the pipeline then just shows the still avatar instead of crashing.

3.6 Rendering the talking clip — `render_talking_clip(...)` (lines ~901–995)

Given a cached avatar image and a synthesised `audio_path`, produce the final `.mp4`. Three modes, chosen by `lipsync_engine`:

<code>lipsync_engine</code>	Pipeline	Speed
"wav2lip_only"	Wav2Lip runs straight on the still image. Lips move, head is static. No SadTalker.	fastest

lipsync_engine	Pipeline	Speed
"sadtalker"	SadTalker head motion + lip motion. No Wav2Lip pass.	medium
"wav2lip" (default)	SadTalker head motion then a Wav2Lip refinement pass.	slowest

For the SadTalker modes there are two execution paths:

- **Warm, in-process (default, SADTALKER_WARM != 0):** calls `sadtalker_engine.generate(...)` in a background thread. The models stay loaded, so a request skips the ~10–20 s of re-loading checkpoints that the subprocess path pays every time.
- **Fresh subprocess (SADTALKER_WARM=0):** shells out to `python inference.py ...`. This was the original design (chosen because on Apple's MPS a fresh process was oddly *faster*); on CUDA the warm path wins.

If the warm path throws (e.g. a broken dependency), it automatically falls back to the subprocess for that request.

3.7 The HTTP endpoints

Method & path	Purpose
GET /api/health	server status + warming_up flag + current warm-up stage
GET /api/voices	list of VieNeu preset voices (name, gender, region)
GET /api/avatars	preset avatar images on the server
POST /preprocess_avatar	upload/pick an avatar → returns a background-removed PNG URL; the frontend shows it as the "standby" image
POST /generate	non-streaming full pipeline: text/audio → reply → TTS → render → one .mp4. Returns a JSON with <code>video_url</code> .
POST /generate_stream	streaming (Server-Sent Events) version. The frontend's main path. See below.
POST /agent/chat	a self-contained "chat" endpoint (message + persona + history → avatar video).
POST /api/custom_voice/upload	upload an audio sample → stored as a VoxCPM clone reference, returns a <code>ref_id</code>
GET /api/history / POST /api/history/clear	read / wipe a session's conversation memory
POST /api/elevenlabs/*	list voices, design a voice, save a designed voice, speech-to-speech "voice changer"
POST /api/push_video, /api/push_video/start, /stop, /status	the "dial out to a listening device on port 9999" mechanism (see §5)

POST /generate_stream in detail (lines ~1193–1375)

1. Resolves the avatar image and the text to speak (running Gemini/Claude if `use_gemini=true`).

2. Background-removes the avatar, caches it, builds the "freeze" clip (disabled by default now).
3. Opens an **SSE stream** and, inside it:
4. Renders the reply as **one clip** (the text is *not* split into per-sentence clips any more — that caused playback to stop after the first sentence).
5. `yields` a `meta` event, then a `clip` event with the video URL, then `done`.
6. Errors are sent as `{"type": "error"}` events, not HTTP errors.
7. After the last clip, copies it to `result/latest_result.mp4` and spawns a thread that calls `socket_video_server.broadcast_video_update(...)` — this is what pushes to the fan bridge.

3.8 Warm-up & the readiness gate (lines ~496–566)

- `@app.on_event("startup") _warmup_models()` — a background thread that pre-loads the heavy models so the *first* user request doesn't stall. `WARMUP=all` (default) loads SadTalker + VieNeu + VoxCPM + Whisper + the AI client; `WARMUP=off` disables it; or give a comma list like `WARMUP=vieneu,whisper`. It sets `_warm_state["ready"] = True` when finished.
- `@app.middleware("http") _warmup_gate(...)` — while `ready` is `False`, any request to `/generate*`, `/agent/chat`, `/preprocess_avatar`, `/api/custom_voice` returns **HTTP 503** with `{"warming_up": true, "stage": "..."}.` The frontend polls `/api/health`, shows a "Setting up models..." overlay, and disables the Generate button until it clears.

3.9 Config reference (environment variables in `.env`)

Variable	Meaning	Default
<code>AI_PROVIDER</code>	<code>gemini</code> or <code>claude</code>	<code>gemini</code>
<code>GEMINI_API_KEY</code>	Gemini key	—
<code>GEMINI_MODEL</code>	pin one Gemini model	(tries <code>lite</code> → <code>latest</code> → <code>flash</code>)
<code>ANTHROPIC_API_KEY</code> / <code>ANTHROPIC_WORKSPACE_ID</code>	Claude key (+ workspace id if scoped)	—
<code>CLAUDE_MODEL</code> / <code>CLAUDE_EFFORT</code>	Claude model / reasoning effort	<code>claude-opus-5</code> / <code>low</code>
<code>AI_BRIEF</code>	<code>1</code> → force one short sentence	<code>off</code>
<code>AI_MAX_TOKENS</code>	reply length cap	<code>512</code>
<code>CLAUDE_HISTORY_TURNS</code>	memory messages sent to the model	<code>20</code>
<code>GEMINI_STT_FIRST</code>	transcribe locally, send text to Gemini (faster on weak links)	<code>off</code>
<code>STT_ENGINE</code> / <code>WHISPER_MODEL</code> / <code>STT_DEVICE</code>	speech-to-text backend / model / device	<code>faster-whisper</code> / <code>large-v3</code> / <code>auto</code>
<code>WARMUP</code>	which models to preload at boot	<code>all</code>
<code>SADTALKER_WARM</code>	keep SadTalker in-process (0 = subprocess)	<code>1</code>

Variable	Meaning	Default
SADTALKER_BATCH_SIZE / WAV2LIP_BATCH_SIZE	render batch sizes (VRAM vs speed)	4 / 32
TTS_DEVICE	cuda / cpu / mps for TTS	auto
SADTALKER_FORCE_CPU	force the renderer onto CPU	off
FORCE_IPV4	make all outbound HTTP use IPv4 (fixes Gemini "read timed out" on broken IPv6)	off
PUBLIC_BASE_URL	URL other devices use to reach the server	http://127.0.0.1:8000
ENABLE_FREEZE_FRAME	re-enable the frozen-avatar clip while rendering	off
REMOVE_BG_API_KEY	remove.bg fallback for background removal	—
ELEVENLABS_API_KEY	ElevenLabs features	—

4. `sadtalker_engine.py` — the warm renderer

~120 lines. Its whole reason to exist: SadTalker's `inference.py` loads **every checkpoint from disk to GPU on every run** (~10–20 s). This module loads them **once**.

- `_get_models(preprocess)` — loads and caches the three SadTalker model groups (`CropAndExtract`, `Audio2Coeff`, `AnimateFromCoeff`) keyed by the `preprocess` mode. A `threading.Lock` serialises access because the models can't run two generations at once.
- `warmup(preprocess)` — called at server start to pre-load.
- `generate(source_image, audio_path, result_dir, clip_name, ...)` — runs the exact same pipeline `inference.py` does (3D-face extraction → audio-to-expression coefficients → face render), but with the pre-loaded models, and moves the result to `result_dir/clip_name`.
- Sets `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` to reduce GPU memory fragmentation.

Note on OOM: on a 16 GB GPU, `WARMUP=all + SADTALKER_WARM=1 + VoxCPM + Whisper-medium` can consume ~14 GB, leaving nothing for the Wav2Lip pass. For a Gemini + `wav2lip_only` setup the recommended `.env` is `WARMUP=vieneu, SADTALKER_WARM=0, TTS_DEVICE=cpu`.

5. The socket push layer

The fan bridge receives videos over a tiny custom TCP protocol: an **8-byte big-endian length**, then that many bytes of MP4.

`backend/socket_video_server.py` (112 lines)

- Runs a listener on `0.0.0.0:9999` in a background thread (started from `server_api.py` at import time).
- Keeps a list of connected client sockets (`active_clients`).

- `broadcast_video_update(file_path)` — sends the length header + file bytes to **every** connected client. Called by `server_api.py` after each generation.
- When a new client connects, it's immediately sent the current `latest_result.mp4`.

Direction: the backend **listens**, the fan bridge **connects in**. This is the path used automatically after every generation.

`backend/socket_video_client.py` / `holofan_test_receiver.py`

Reference receivers. `socket_video_client.py` connects to `<backend>:9999` and saves each pushed video to a file, reconnecting on drop. `holofan_test_receiver.py` is the same idea but timestamps each transfer for latency measurement.

`/api/push_video/*` (the *other* direction)

These endpoints make the backend **dial out** to `target_ip:9999` and expect *that* device to be listening. This is a different topology (used when the fan itself, or a box on the fan's LAN, listens). **It is not used with**

`holofan_autopush.py` — that connects inward, so the auto-broadcast path handles everything and the "Stream to device" toggle in the UI should be left off.

6. The fan bridge — `holofan_autopush.py`

~260 lines. Runs on the machine physically next to the fan (Mac or Windows).

Two jobs

Job 1 — receive (`receive_loop`) Connects to `<backend>:9999`, reads the 8-byte length + MP4 for each push, saves it as `received_holofan_video.mp4` (overwritten each time), reconnects automatically if the tunnel drops.

Job 2 — drive Holoscope (`upload_to_holoscope`) Holoscope has no API, so this uses `pyautogui` to physically click through it. Buttons are found by **image matching** (`<name>.png` files cropped from the real screen) with a **window-relative coordinate fallback** if the PNG is missing. The window is located with `osascript` (macOS) or `pygetwindow` (Windows).

- **Windows path (`_upload_windows`)** — the good one. Windows Holoscope has a real file dialog: Transfer → type the file path → Open → "Start transfer" → "Confirm" → progress to 100%.
- **macOS path (`_upload_mac`)** — the wrapped iPhone app. Imports the clip into the macOS **Photos** library via AppleScript (its "Video" picker reads Photos), then Transfer → Video → newest thumbnail → Confirm. **Fragile** — the wrapped app's Photos picker is often broken; the Windows version is strongly preferred.

Why the Mac struggles

`Holoscope.app` on an Apple-Silicon Mac is an **iPhone app running in a compatibility layer**. Its "Video" picker (an iOS photo picker) frequently does nothing, and its "Local" tab only shows Holoscope's own internal files. The Windows desktop version has a normal file browser and works cleanly.

7. Startup — `start_holo_mac.sh`

One command on the Mac. It: 1. Opens the **SSH tunnel** to the remote GPU box (`i86-Server`), forwarding ports `8000` and `9999` to `localhost`, and waits until `/api/health` answers. 2. Starts the **frontend** dev server (`npm run dev -- --host`). 3. Starts `holofan_autopush.py 127.0.0.1`. 4. `Ctrl+C` stops all three.

The backend itself (`python server_api.py`) is started separately by SSH-ing into `i86`.

8. The frontend (frontend/src/)

A Vite + React + Tailwind single-page app.

api.js

```
export const API_BASE =
  import.meta.env.VITE_API_BASE || `http://${window.location.hostname}:8000`;
```

So when you open the page at `http://localhost:5173` the API is `http://localhost:8000`; open it from another device by IP and the API follows automatically.

App.jsx (~466 lines) — holds all the state and the generation logic

- **getSessionId()** — a random id stored in `localStorage`, sent with every request so the backend's conversation memory is stable per browser.
- **State** — the input text, active tab (`TEXT / AUDIO / LIVE_MIC`), selected avatar, TTS engine & voice, `useGemini` (AI-reply toggle), persona, `lipsyncEngine`, plus generation state (`isGenerating`, `generatedVideoUrl`, ...).
- **Warm-up poll** — `useEffect` hits `/api/health` every 2 s; while `warming_up` is true it shows a full-screen "Setting up models..." overlay and `isGenerateReady` is forced false.
- **processAvatarAndGenerateIdle()** — on picking/uploading an avatar, POSTs to `/preprocess_avatar` and stores the returned background-removed image URL as the standby image.
- **handleGenerate()** — the core. Builds a `FormData`:
 - `TEXT` tab, or `LIVE_MIC` with a typed message → `inputType=text`. If the AI toggle is on (always on for live-mode typing), `use_gemini=true` and the typed text is treated as a **message to the AI**; otherwise it's spoken **verbatim**.
 - `AUDIO / LIVE_MIC` with a recording → `inputType=audio` + the audio file.
 - always attaches `session_id`, voice settings, `lipsync_engine`, etc.
 - `fetch("/generate_stream")` and reads the **SSE stream**: on each `clip` event it sets `generatedVideoUrl` (the `<video>` autoplays); `error` events surface a message; `done` ends it.
- **clearConversation()** — POSTs `/api/history/clear` for this session.

components/Sidebar/

- **Sidebar.jsx** — lays out the avatar picker + input area; passes ~40 props through.
- **InputArea.jsx** (~660 lines) — the tabbed input:
 - `TEXT` — a script box + the "■ AI Reply (chat, remembers)" toggle + persona field + "Clear memory".
 - `LIVE_MIC` — record from the microphone (`MediaRecorder`), **or** type a message to the AI; the AI toggle + persona live here too.
 - `AUDIO` — upload an audio file.
- **VoiceEngineSelector** (shared) — `VieNeu` vs `VoxCPM`; for `VoxCPM` the clone source is `ElevenLabs / VieNeu` preset / **Upload Voice** (posts to `/api/custom_voice/upload`); also the `ElevenLabs` "voice changer".
- **SadTalkerSettings.jsx** — lip-sync engine (`wav2lip / sadtalker / wav2lip_only`), preprocess mode, enhancer, expression scale.
- **CharacterList.jsx / UploadArea.jsx** — preset avatars grid / drag-and-drop upload.

components/VideoArea/

- **MainPlayer.jsx** — the `<video>` element. When there's a generated clip it plays it; otherwise it shows the static standby avatar image. Status badges: "LIVE STANDBY", "LISTENING & PROCESSING", "LIVE AVATAR TALKING...". A processing overlay covers it while `isGenerating`.
- **VideoArea.jsx** — wraps the player, the transcript box, recent clips, and the (unused) "Socket Video Sender" panel.
- **RecentVideos.jsx** — thumbnails of the last few results.

9. The third-party model code (treated as libraries)

You don't need to read these to understand the project, but here's what they are:

- **src/utils/preprocess.py (CropAndExtract)** — detects the face, crops it, and extracts **3DMM coefficients** (a parametric description of the face's shape and expression).
 - **src/test_audio2coeff.py (Audio2Coeff)** — the SadTalker network that maps the driving audio to per-frame **expression + head-pose coefficients**.
 - **src/facerender/animate.py (AnimateFromCoeff)** — renders the actual video frames from the avatar image + those coefficients (this is the heavy GPU step). Can optionally run **GFPGAN** to sharpen faces (the enhancer option — slow, off by default).
 - **models/wav2lip.py + src/utils/wav2lip_inference.py** — the Wav2Lip network: given a face video (or still image) + audio, it re-draws the mouth region frame-by-frame to match the speech. `wav2lip_processor.py` is a ~35-line wrapper that runs it as a subprocess and falls back to the input video on any error.
 - **face_detection/** — an S3FD face detector used by Wav2Lip.
 - **inference.py** — SadTalker's original CLI. Still used as the fallback path when `SADTALKER_WARM=0`. It also has a small local patch: `SADTALKER_FORCE_CPU=1` forces `args.device = "cpu"`.
-

10. End-to-end: what happens when you click "Generate"

Assume: backend up on i86, `start_holo_mac.sh` running on the Mac, Holoscope open with the fan connected, `AI_PROVIDER=gemini`, `lipsync_engine=wav2lip_only`, `AI_BRIEF=1`, `LIVE_MIC` tab with a typed message.

1. **Browser** — `handleGenerate()` builds `FormData` (`inputType=text`, `use_gemini=true`, `text=...`, `session_id`, `voice_name`, `lipsync_engine=wav2lip_only`, ...) and fetches `POST http://localhost:8000/generate_stream`. The request travels through the **SSH tunnel** to `i86:8000`.
2. **Backend** — `generate_video_stream()` runs:
3. resolves the avatar (already preprocessed), background-removes / caches it.
`generate_gemini_response(user_message=text, session_id=...)`:
 - builds the "speak-aloud, one short sentence" system prompt + persona.
 - loads `conversations/<session_id>.json` (the memory).
 - `_gemini_reply(...)` → `POST` to Gemini `gemini-2.5-flash-lite` with history + the message → a one-sentence Vietnamese reply.
 - appends `{user}` and `{assistant}` to the history file.
5. `synthesize_tts(reply, audio_path, tts_engine="vietneu", voice_name="Thái S...")` → a `.wav` on the GPU box.
6. `render_talking_clip/avatar, audio_path, ..., lipsync_engine="wav2lip_only")` → `process_wav2lip()` shells out to Wav2Lip, which loops the still avatar frame over the audio and re-draws the mouth → `clip_0.mp4`.
7. copies it to `result/latest_result.mp4`; SSE-emits a `clip` event with its URL; spawns a thread → `socket_video_server.broadcast_video_update("result/latest_result.mp4")`.
8. **Browser** — the SSE reader gets the `clip` event, sets `generatedVideoUrl` (`http://localhost:8000/result/...` through the tunnel), the `<video>` autoplays the talking avatar.
9. **socket_video_server** — sends `[8-byte length][mp4 bytes]` to every client on port 9999. That reaches `localhost:9999` on the Mac → the tunnel → `i86:9999`.
10. **holofan_autopush.py** — `receive_loop` reads the length + bytes, writes `received_holofan_video.mp4`, then `upload_to_holoscope()`:
11. (Windows) focuses Holoscope, clicks **Transfer**, types `C:\...\received_holofan_video.mp4` into the file dialog, Enter, clicks **Start transfer**, clicks **Confirm** → the % bar climbs.
12. (Mac) imports to Photos, then Transfer → Video → newest → ■■.
13. **Holoscope** → **fan** — over the fan's `3D-P...` Wi-Fi hotspot, the clip lands in a playlist slot and the fan spins it up as a hologram.

Total time on a warm CUDA box: roughly **6–10 s** (Gemini ~1–2 s, TTS ~0.5 s, Wav2Lip render ~3–4 s, transfer ~1–2 s), plus a few seconds for Holoscope's own upload to the fan.

11. Known constraints & gotchas

- **The render is not real-time.** ~1–2× the audio length even fully optimised. Sub-5-second latency would need a different (streaming) talking-head model.
- **Reply length drives everything** — a 13 s reply is ~2.5× the render of a 5 s one. `AI_BRIEF=1` keeps it short.
- **Two networks, one machine** — the fan bridge needs internet (for the tunnel) *and* the fan's Wi-Fi at once. Solutions: Ethernet + Wi-Fi, or iPhone USB tethering for internet with Wi-Fi free for the fan.
- **Holoscope on Mac** is a wrapped iOS app with a flaky Photos picker; the Windows version is the reliable one.
- **Holoscope has no network API** — the fan connects *out* to Holoscope on port 6666 with an undocumented protocol, so a direct push (skipping Holoscope) would require reverse-engineering it.
- **Fan needs a pure black background** and clips **under 1 minute**; Holoscope's transfer screen also trims to the first 10 s.
- **GPU memory** — don't `WARMUP=all` a 16 GB card if you also render with Wav2Lip; trim the warm set.